

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2017 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Varianta 3 (12 iulie 2017)	2
SUBIECTUL I (30 de puncte)	2
1. Algoritmica submulțimilor	2
2. Proiectare baze de date: Normalizare	2
a) Noțiuni preliminare în modelarea conceptuală	2
b) Cele trei Forme Normale (1FN, 2FN, 3FN)	2
1. Prima Formă Normală (1FN)	2
2. A Doua Formă Normală (2FN)	3
3. A Treia Formă Normală (3FN)	3
SUBIECTUL al II-lea (30 de puncte)	3
1. Programare: Eliminarea nod din listă dublu înălțuită	3
2. Algoritm eficient: Optimizare submulțime cu sume distincte	6
SUBIECTUL al III-lea (30 de puncte)	9
1. Itemi pentru secvența A: proprietăți ale secvențelor de valori	9
2. Metodă didactică pentru secvența B: tabele în editorul de texte	9

I. Rezolvare Titularizare Varianta 3 (12 iulie 2017)

[Subiect PDF](file:

SUBIECTUL I (30 de puncte)

1. Algoritmica submulțimilor

- **Backtracking:** Generarea tuturor submulțimilor unei mulțimi de N elemente prin vectori binari de lungime N (unde $x_i = 1$ dacă elementul i aparține submulțimii, și 0 altfel).
- **Complexitate:** $O(2^N)$ în timp.
- **Problemă (Generare submulțimi):**

```
#include <iostream>
using namespace std;
int x[20], n;
void afisare() {
    cout << "{ ";
    for (int i = 1; i <= n; ++i) if (x[i] == 1) cout << i << " ";
    cout << "}\n";
}
void backtrack(int k) {
    if (k > n) { afisare(); return; }
    for (int val = 0; val <= 1; ++val) {
        x[k] = val;
        backtrack(k + 1);
    }
}
```

2. Proiectare baze de date: Normalizare

a) Noțiuni preliminare în modelarea conceptuală

1. **Model conceptual:** O reprezentare abstractă, formală și independentă de implementarea hardware sau software a datelor dintr-o problemă din lumea reală, axată pe entități și asocieri logice.
2. **Entitate:** O clasă de obiecte, concepte, persoane sau evenimente distincte din lumea reală despre care dorim să stocăm date (ex. STUDENT, CURS).
3. **Instanță:** Un element individual concret, o înregistrare specifică aparținând unei entități (ex. studentul cu numele „Popescu Ion”).
4. **Atribut:** O proprietate sau caracteristică individuală care descrie o entitate (ex. nume, data_nasterii, pret).
5. **Identificator unic (Cheie primară - PK):** Un atribut sau grup de attribute ale cărui valori identifică în mod unic și neambiguu fiecare instanță a unei entități (ex. CNP sau id_student).
6. **Relație:** O legătură sau asociere logică stabilită între două sau mai multe entități (ex. relația înscris la între STUDENT și CURS), caracterizată prin cardinalitate (1:1, 1:M, M:N).

b) Cele trei Forme Normale (1FN, 2FN, 3FN)

1. Prima Formă Normală (1FN)

- **Descriere:** O entitate/tabelă este în 1FN dacă și numai dacă toate attributele sale conțin exclusiv valori atomice (inseparabile), iar tabela nu conține grupuri repetitive de attribute sau liste de valori.

- **Exemplu de încălcare a 1FN:** Tabela ANGAJAT conține atributele: (id_angajat, nume, proiecte_alocate). Instanța: (1, 'Popescu Ion', 'Proiect A, Proiect B, Proiect C'). Câmpul proiecte_alocate conține valori neatomice (o listă de proiecte).
- **Soluție de normalizare (respectare 1FN):** Fiecare intersecție rând-coloană trebuie să conțină o singură valoare atomică. Putem crea o structură în care fiecare proiect are rândul său: ANGAJAT_PROIECT(id_angajat, nume, proiect_alocat) Instanțe:
 - (1, 'Popescu Ion', 'Proiect A')
 - (1, 'Popescu Ion', 'Proiect B')
 - (1, 'Popescu Ion', 'Proiect C')

2. A Doua Formă Normală (2FN)

- **Descriere:** O entitate este în 2FN dacă este în 1FN și toate atributele non-cheie sunt complet dependente funcțional de întreaga cheie primară (nu doar de o parte a acesteia, în cazul cheilor primare compuse). 2FN elimină dependențele parțiale.
- **Exemplu de încălcare a 2FN:** Fie tabela PROIECT_STUDENT cu cheia primară compusă (id_student, id_proiect) și atributele: (id_student, id_proiect, nota_obtinuta, denumire_proiect). Aici, nota_obtinuta depinde de întreaga cheie (student + proiect). Însă, denumire_proiect depinde funcțional doar de id_proiect (dependență parțială de o parte a cheii).
- **Soluție de normalizare (respectare 2FN):** Se separă atributele dependente parțial în tabele distincte:
 - EVALUARE_STUDENT(id_student, id_proiect, nota_obtinuta) - unde cheia primară este (id_student, id_proiect).
 - PROIECT(id_proiect, denumire_proiect) - unde cheia primară este id_proiect.

3. A Treia Formă Normală (3FN)

- **Descriere:** O entitate este în 3FN dacă este în 2FN și nu conține dependențe tranzitive ale atributelor non-cheie față de cheia primară. Altfel spus, niciun atribut non-cheie nu trebuie să depindă de cheia primară prin intermediul unui alt atribut non-cheie.
- **Exemplu de încălcare a 3FN:** Fie tabela ANGAJAT cu cheia primară id_angajat și atributele: (id_angajat, nume, id_departament, nume_departament). Aici, id_angajat determină id_departament, iar id_departament determină nume_departament. Avem o dependență tranzitivă: id_angajat → id_departament → nume_departament.
- **Soluție de normalizare (respectare 3FN):** Se izolează atributele cu dependență tranzitivă în tabele separate:
 - ANGAJAT(id_angajat, nume, id_departament) - unde id_departament devine cheie străină (FK).
 - DEPARTAMENT(id_departament, nume_departament) - unde cheia primară este id_departament.

—

SUBIECTUL al II-lea (30 de puncte)

1. Programare: Eliminare nod din listă dublu înlănțuită

Soluție C++:

```
#include <iostream>
#include <vector>
using namespace std;
```

```

struct Nod {
    int info;
    Nod *prec, *urm;
};

void elimin(Nod* &p, Nod* q) {
    if (q == nullptr) return;
    if (q->prec != nullptr) q->prec->urm = q->urm;
    if (q->urm != nullptr) q->urm->prec = q->prec;
    if (q == p) p = q->urm;
    delete q;
}

int main() {
    int val;
    vector<int> vals;
    while (cin >> val && val != 0) {
        vals.push_back(val);
    }
    if (vals.empty()) {
        cout << "nu exista\n";
        return 0;
    }

    int last_val = vals.back();
    Nod *p = nullptr, *u = nullptr;
    for (int x : vals) {
        Nod* nou = new Nod{x, u, nullptr};
        if (p == nullptr) { p = nou; u = nou; }
        else { u->urm = nou; u = nou; }
    }

    Nod* curr = p;
    while (curr != nullptr) {
        Nod* urm = curr->urm;
        if (curr->info == last_val) {
            elimin(p, curr);
        }
        curr = urm;
    }

    if (p == nullptr) {
        cout << "nu exista\n";
    } else {
        for (Nod* c = p; c != nullptr; c = c->urm) {
            cout << c->info << (c->urm == nullptr ? "" : " ");
        }
        cout << endl;
    }
    return 0;
}

```

Soluție Pascal:

```

program EliminareNoduri;
type

```

```

PNod = ^TNod;
TNod = record
    info: integer;
    prec, urm: PNod;
end;
var
    p, u, nou, curr, urm_nod: PNod;
    val, last_val, i: integer;
    vals: array[1..100] of integer;
    count: integer;

procedure elimin(var p_list: PNod; q: PNod);
begin
    if q = nil then exit;
    if q^.prec <> nil then q^.prec^.urm := q^.urm;
    if q^.urm <> nil then q^.urm^.prec := q^.prec;
    if q = p_list then p_list := q^.urm;
    dispose(q);
end;

begin
    count := 0;
    read(val);
    while val <> 0 do
        begin
            count := count + 1;
            vals[count] := val;
            read(val);
        end;

        if count = 0 then
            begin
                writeln('nu exista');
                exit;
            end;

        last_val := vals[count];
        p := nil; u := nil;
        for i := 1 to count do
            begin
                new(nou);
                nou^.info := vals[i];
                nou^.prec := u;
                nou^.urm := nil;
                if p = nil then
                    begin
                        p := nou; u := nou;
                    end
                else
                    begin
                        u^.urm := nou;
                        u := nou;
                    end;
            end;
        end;

        curr := p;
    
```

```

while curr <> nil do
begin
    urm_nod := curr^.urm;
    if curr^.info = last_val then
        elimin(p, curr);
    curr := urm_nod;
end;

if p = nil then
    writeln('nu exista')
else
begin
    curr := p;
    while curr <> nil do
    begin
        write(curr^.info);
        if curr^.urm <> nil then write(' ');
        curr := curr^.urm;
    end;
    writeln;
end;
end.

```

2. Algoritm eficient: Optimizare submulțime cu sume distincte

Soluție C++:

```

#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ifstream fin("titu.in");
    int n, k;
    if (!(fin >> n >> k)) return 0;
    vector<long long> even, odd;
    for (int i = 0; i < n; ++i) {
        long long val;
        fin >> val;
        if (val % 2 == 0) even.push_back(val);
        else odd.push_back(val);
    }
    fin.close();

    sort(even.rbegin(), even.rend());
    sort(odd.begin(), odd.end());

    int num_even = even.size();
    int num_odd = odd.size();
    vector<long long> pref_odd(num_odd + 1, 0);
    for (int i = 0; i < num_odd; ++i) pref_odd[i+1] = pref_odd[i] + odd[i];

    int max_total = 0;
    long long sum_even = 0;

```

```

int limit = min(k, num_even);
for (int e = 1; e <= limit; ++e) {
    sum_even += even[e-1];
    int st = 0, dr = num_odd, best_j = 0;
    while (st <= dr) {
        int mid = st + (dr - st) / 2;
        if (pref_odd[mid] < sum_even) {
            best_j = mid;
            st = mid + 1;
        } else {
            dr = mid - 1;
        }
    }
    max_total = max(max_total, e + best_j);
}
cout << max_total << endl;
return 0;
}

```

Soluție Pascal:

```

program MaxElementeSelectate;
var
    fin: text;
    n, k, i, val, limit: integer;
    even, odd: array[1..10000] of int64;
    pref_odd: array[0..10000] of int64;
    num_even, num_odd: integer;
    sum_even: int64;
    max_total, e_val, st, dr, mid, best_j: integer;

procedure QuickSortEven(l, r: integer);
var i_idx, j_idx: integer; pivot, tmp: int64;
begin
    i_idx := l; j_idx := r; pivot := even[(l + r) div 2];
    repeat
        while even[i_idx] > pivot do i_idx := i_idx + 1;
        while even[j_idx] < pivot do j_idx := j_idx - 1;
        if i_idx <= j_idx then
            begin
                tmp := even[i_idx]; even[i_idx] := even[j_idx]; even[j_idx] := tmp;
                i_idx := i_idx + 1; j_idx := j_idx - 1;
            end;
    until i_idx > j_idx;
    if l < j_idx then QuickSortEven(l, j_idx);
    if i_idx < r then QuickSortEven(i_idx, r);
end;

procedure QuickSortOdd(l, r: integer);
var i_idx, j_idx: integer; pivot, tmp: int64;
begin
    i_idx := l; j_idx := r; pivot := odd[(l + r) div 2];
    repeat
        while odd[i_idx] < pivot do i_idx := i_idx + 1;
        while odd[j_idx] > pivot do j_idx := j_idx - 1;
        if i_idx <= j_idx then
            begin

```

```

    tmp := odd[i_idx]; odd[i_idx] := odd[j_idx]; odd[j_idx] := tmp;
    i_idx := i_idx + 1; j_idx := j_idx - 1;
end;
until i_idx > j_idx;
if l < j_idx then QuickSortOdd(l, j_idx);
if i_idx < r then QuickSortOdd(i_idx, r);
end;

begin
assign(fin, 'titu.in'); reset(fin);
read(fin, n, k);
num_even := 0; num_odd := 0;
for i := 1 to n do
begin
    read(fin, val);
    if val mod 2 = 0 then
begin
        num_even := num_even + 1; even[num_even] := val;
    end
    else
begin
        num_odd := num_odd + 1; odd[num_odd] := val;
    end;
end;
close(fin);

if num_even > 0 then QuickSortEven(1, num_even);
if num_odd > 0 then QuickSortOdd(1, num_odd);

pref_odd[0] := 0;
for i := 1 to num_odd do pref_odd[i] := pref_odd[i - 1] + odd[i];

max_total := 0; sum_even := 0;
if k < num_even then limit := k else limit := num_even;
for e_val := 1 to limit do
begin
    sum_even := sum_even + even[e_val];
    st := 0; dr := num_odd; best_j := 0;
    while st <= dr do
begin
        mid := (st + dr) div 2;
        if pref_odd[mid] < sum_even then
begin
            best_j := mid; st := mid + 1;
        end
        else dr := mid - 1;
    end;
    if e_val + best_j > max_total then max_total := e_val + best_j;
end;
writeln(max_total);
end.

```

—

SUBIECTUL al III-lea (30 de puncte)

1. Itemi pentru secvența A: proprietăți ale secvențelor de valori

- **Obiectiv:** Alegere multiplă. Un număr perfect este un număr egal cu: A. suma cifrelor sale B. suma divizorilor proprii C. produsul divizorilor D. numărul de cifre. **Răspuns:** B.
- **Semiobiectiv:** Completați: pentru a verifica dacă toate elementele unei secvențe au proprietatea P, inițializăm un indicator cu valoarea [spațiu liber]. **Răspuns:** adevărat/true.
- **Subiectiv:** Scrieți un algoritm care verifică dacă toate elementele unui vector sunt numere perfecte. **Răspuns:** parcurgere, calcul sumă divizori proprii, oprire la primul contraexemplu.

2. Metodă didactică pentru secvența B: tabele în editorul de texte

Metodă: demonstrația practică urmată de exercițiu. **Caracteristici:** profesorul arată procedura; elevii reproduc și adaptează; feedbackul este imediat. **Argumente:** conținutul este practic și cere deprinderi de operare.

Activități: inserarea unui tabel pentru orar; formatarea tabelelor prin borduri, îmbinare celule și aliniere.

Scenariu: Profesorul demonstrează inserarea tabelului, elevii creează propriul tabel, apoi aplică operații de formatare. Profesorul verifică lizibilitatea și corectitudinea operațiilor.